COMPUTER NETWORKS FUNDAMENTALS

Chapter 2: The Application Layer

What your apps actually say to each other — and how they find each other first

What We'll Cover





App Principles



The Web & HTTP



State & Speed



DNS



Email



Streaming

BY THE END OF THIS CLASS, YOU WILL BE ABLE TO

1. Explain how two programs on different machines find and talk to each other
2. Read an HTTP request and response, and say what each part is for
3. Trace how muele.mak.ac.ug becomes an IP address — and why that is not one big server



01 Principles of Network Applications

What an app needs before it can say anything

Where We Are: Back to the Top of the Stack



Chapter 1 built the stack from the bottom. Now we start at the top — the layer you actually use.

Recall from Chapter 1: only hosts run the application layer. No router in the middle ever opens your message — it only reads the network header and forwards.

Applications You Already Use

Every one of these is a program at the edge of the network, talking to another program somewhere else.

MUELE

course materials, quizzes, assignment uploads

ACMIS portal

registration, results, fees — myportal.mak.ac.ug

Webmail

your @mak.ac.ug address

WhatsApp

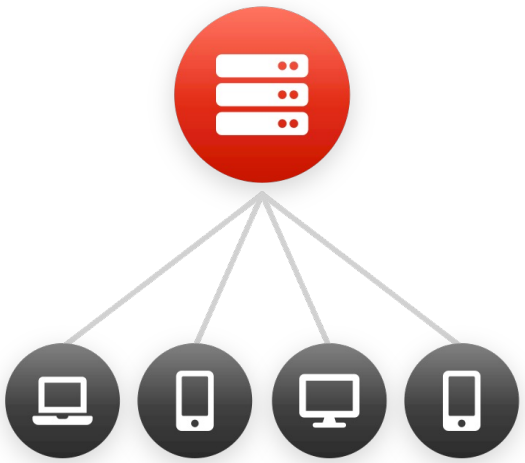
messaging and calls

The key point: you never had to write networking code to build any of these. The application only has to say WHAT it wants — the layers below handle getting it there.

Two Ways to Organise an Application

CLIENT – SERVER

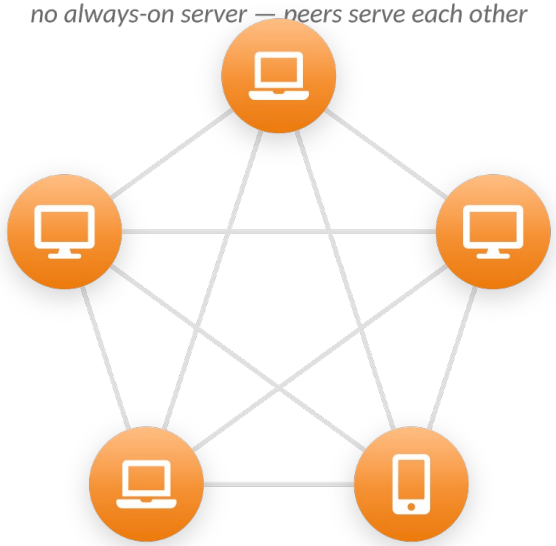
one always-on server, many clients



clients only talk to the server

PEER – TO – PEER

no always-on server — peers serve each other

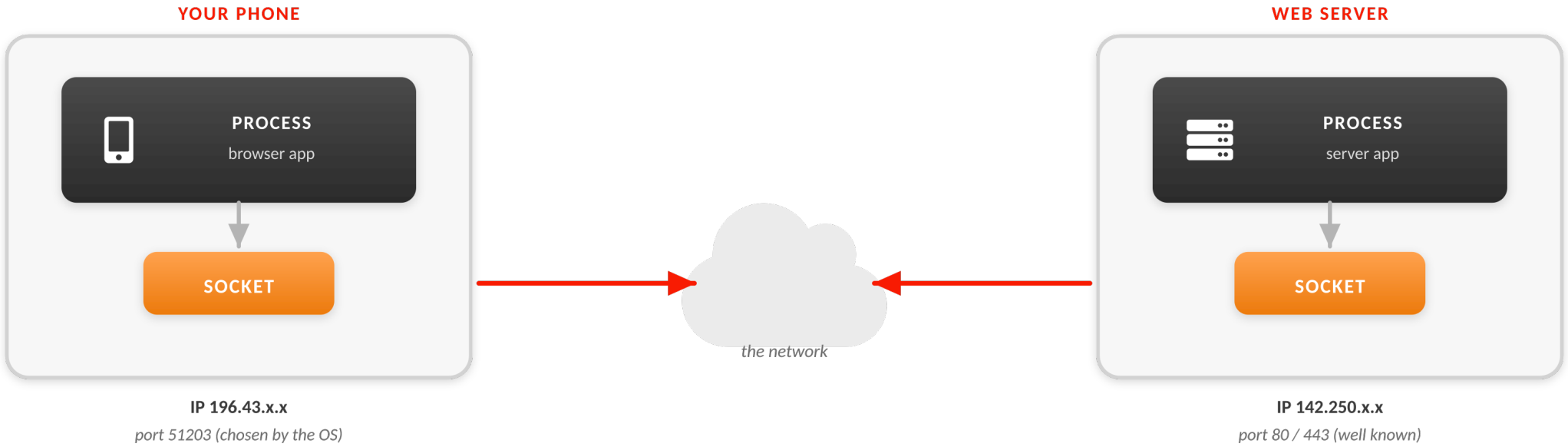


every peer is both client and server

MUELE and ACMIS are client–server: the server is always on, has a fixed address, and every student is a client. WhatsApp calls lean peer-to-peer — your phones talk more directly.

How One Program Reaches Another

Two things are needed: a doorway out of the program (a socket), and a full address (which host, plus which program on it).



A socket is the doorway between a process and the network. An address needs BOTH the IP (which host) and the port (which process).

IP address = which host. Port number = which program on that host. Web servers listen on port 80 (or 443 for HTTPS) so your browser always knows where to knock.

What an Application-Layer Protocol Defines



Same idea as Chapter 1 — an agreed set of rules. At this layer the rules cover four things:



Message types

requests and responses — who may say what



Message syntax

what fields exist, and how they are written down



Field meaning

what the contents of each field actually mean



Rules

when a program sends, and how it must reply

Open vs proprietary: HTTP and SMTP are published openly, so anyone can write a browser or mail client. WhatsApp's protocol is closed — only their apps can speak it.

What Does an App Need From the Network?



Different applications care about very different things. Four dimensions matter:



Reliability

can any data be lost? A file transfer says no. Live voice can tolerate a little.



Throughput

does it need a minimum rate to work at all? Video does. Email does not.



Timing

does data need to arrive quickly? A call needs low delay. A download does not.



Security

must it be private and authenticated? Anything with a password.

Requirements of Familiar Applications



APPLICATION	DATA LOSS	THROUGHPUT	TIME SENSITIVE
Uploading to MUELE	No loss	What is available	Not sensitive
Streaming video	Some loss OK	A steady rate	Sensitive — secs
WhatsApp call	Some loss OK	Modest but steady	Very sensitive — ms
Email	No loss	What is available	Not sensitive
Online game	Some loss OK	Modest	Very sensitive — ms

The Two Transport Services on Offer

The application layer does not build delivery itself — it picks one of two services from the layer below.

	TCP	UDP
Connection first?	Yes — handshake before data	No — just send
Delivery	Reliable: lost data is resent	Best effort: may be lost
Order	Arrives in order	May arrive out of order
Overhead	Higher	Lower — faster, leaner
Used by	Web, email, file transfer	Live video, voice, gaming, DNS

Neither guarantees timing, minimum bandwidth, or security — those must be built on top.

Rule of thumb: if every byte must arrive, choose TCP. If being late is worse than being lost — live voice, video, gaming — UDP is often the better fit.

PRINCIPLES · PART 9

Where Does the Padlock Come From?



Plain TCP carries everything in the clear — anyone on the path could read your MUELE password. TLS is a layer added on top.



HTTP + plain TCP

Everything readable in transit — passwords, messages, page content. Your browser shows "Not secure".



HTTPS = HTTP + TLS

Encrypted so eavesdroppers see nothing useful, the server proves who it is, and tampering is detected.

Check it yourself: muele.mak.ac.ug and myportal.mak.ac.ug both use HTTPS — click the padlock in your browser to see the certificate that proves it.

QUICK CHECK · 1

Your WhatsApp voice call and your MUELE assignment upload — which needs TCP?

- The voice call
- The MUELE upload
- Both need TCP
- Neither — they use UDP



02 The Web and HTTP

What happens when you open MUELE

A Web Page Is Not One Thing

A page is an HTML file that REFERENCES other objects — images, stylesheets, scripts. Each one is fetched separately.

ONE MUELE COURSE PAGE MIGHT BE

1 HTML file
the structure of the page

5 stylesheets & scripts
how it looks and behaves

20+ images & icons
logos, buttons, thumbnails

EVERY OBJECT HAS A URL

`https://muele.mak.ac.ug/course/view.php`

protocol	how to talk — HTTPS
host name	which machine — becomes an IP
path	which object on that machine

This matters: when you open the Network tab of a session (exercise 2) and saw dozens of requests for one page — this is why.

HTTP: The Rules of the Web

HyperText Transfer Protocol — the agreement between every browser and every web server.

Request – response

the client asks for an object; the server sends it back

Runs over TCP

so the page arrives complete and in order

Stateless

the server remembers NOTHING about earlier requests

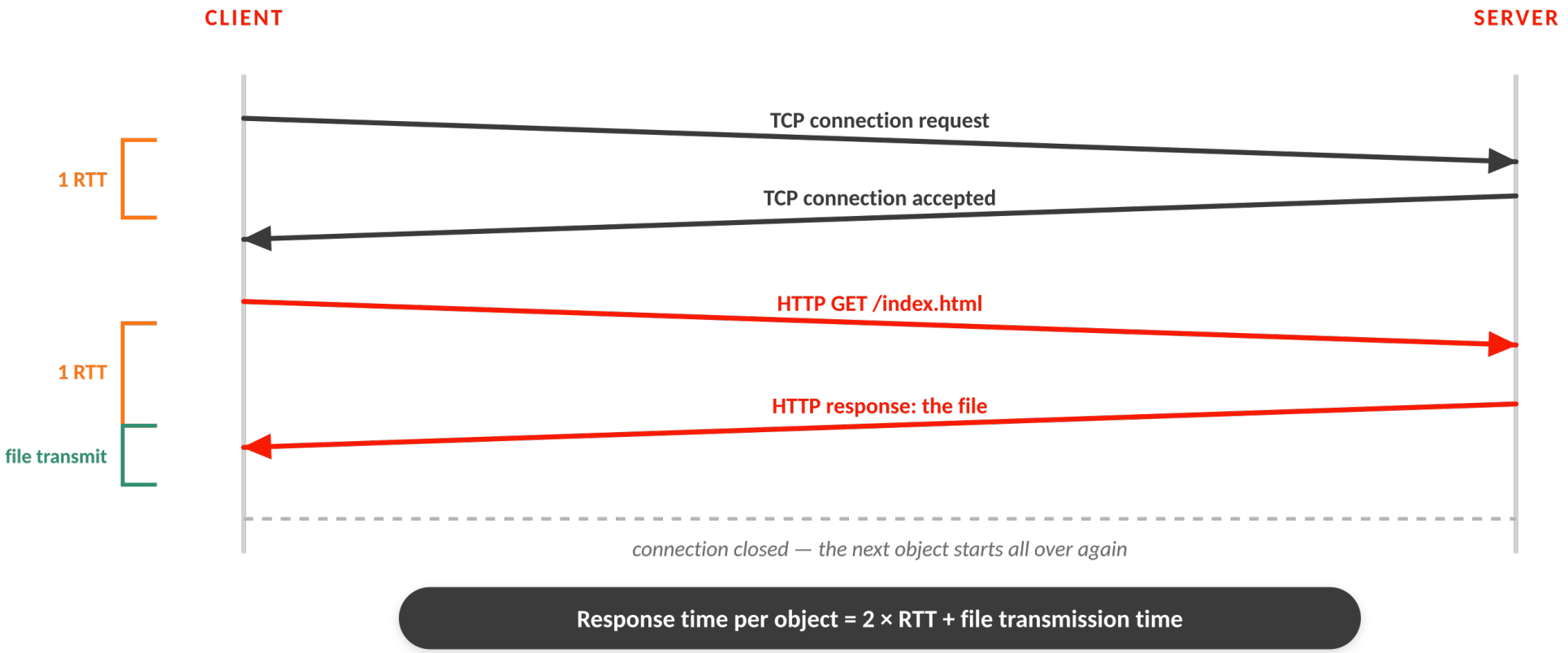
Plain text

the messages are human-readable — you can read them yourself

"Stateless" is the surprising one: MUELE has no memory of you between requests. Every single click arrives as if it were your first. We will see shortly how it fakes a memory.

One Object, One Connection

In the original design, each object needed its own fresh TCP connection — set up, use once, tear down.



Why That Was Slow



Round-trip time (RTT) is how long a small packet takes to go there and come back. Every connection costs two of them before any data flows.

ONE QUICK EXAMPLE

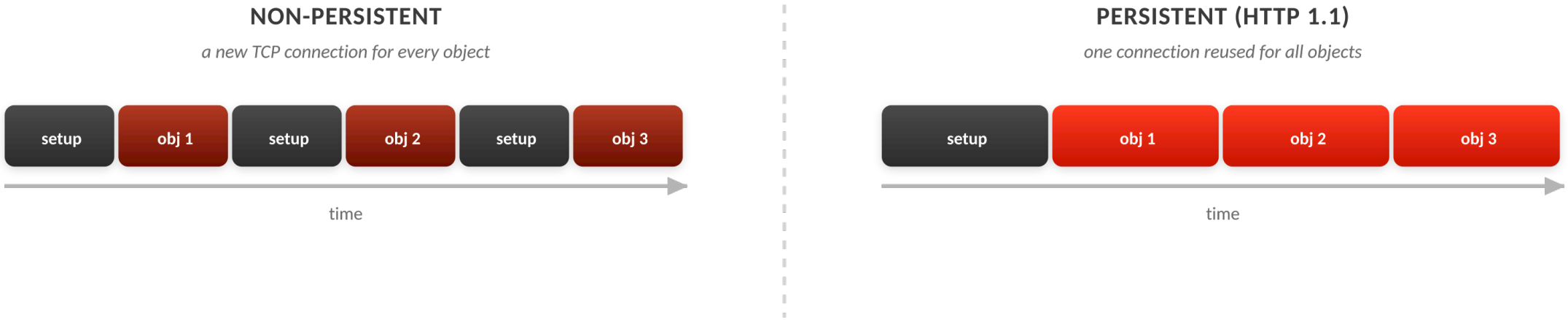
Suppose the RTT to the MUELE server is **40 ms**, and the page needs **1 HTML file + 10 images**.

Each object costs 2 × RTT = 80 ms just in setup and asking.
11 objects × 80 ms = **880 ms** — nearly a full second before you count the data itself.

On a slow or distant link this is very bad. If the RTT were 300 ms — not unusual on a congested mobile connection — the same page would spend over 6 seconds just setting up connections.

The Fix: Keep the Connection Open

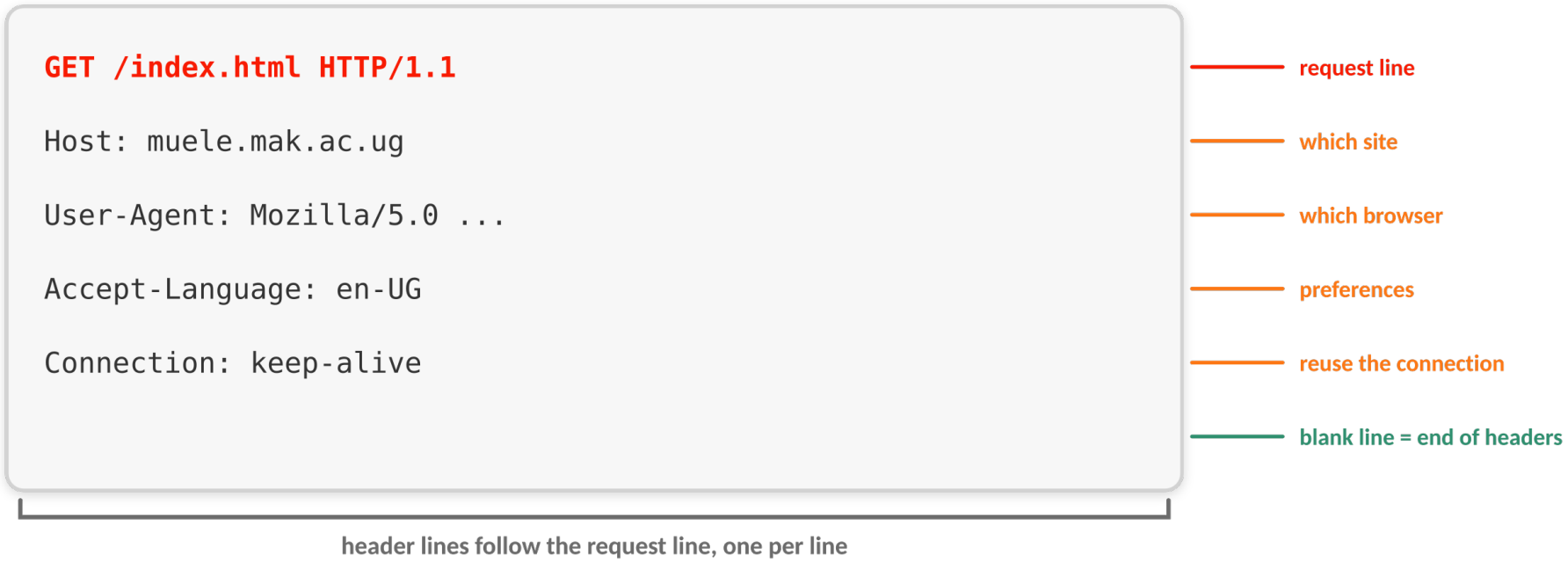
Persistent HTTP — the default since HTTP/1.1 — reuses one connection for every object on the page.



Same three objects — persistent HTTP removes two full connection setups, so the page finishes sooner.

Same eleven objects, one setup instead of eleven. This is the "Connection: keep-alive" header you will see in the next slide.

What Your Browser Actually Sends



Other methods exist too: GET asks for an object, POST submits data (like your MUELE login), PUT uploads, DELETE removes.

What the Server Sends Back



RESPONSE MESSAGE

```
HTTP/1.1 200 OK

Date: Mon, 24 Aug 2026 09:14:02 GMT

Server: nginx

Content-Type: text/html

Content-Length: 6821


<html> ... the actual page ... </html>
```

status line → headers → blank line → the data itself

STATUS CODES YOU WILL MEET

200	**OK** request succeeded
301	**Moved Permanently** the page lives elsewhere now
400	**Bad Request** the server could not understand it
404	**Not Found** no such page on this server
505	**HTTP Version Not Supported** server will not speak this version

See It For Yourself

You already did part of this in the week-2 exercise. Here is what you were looking at:

Browser developer tools

Press F12 → Network tab → reload MUELE. Every row is one request. Click any row to see the exact headers we just studied.

Easiest — start here

Ask the server directly

curl -v https://muele.mak.ac.ug

This prints the request it sent and the response headers it received — the raw protocol.

For the curious

QUICK CHECK · 2

MUELE shows your name on every page. But HTTP is stateless. How?

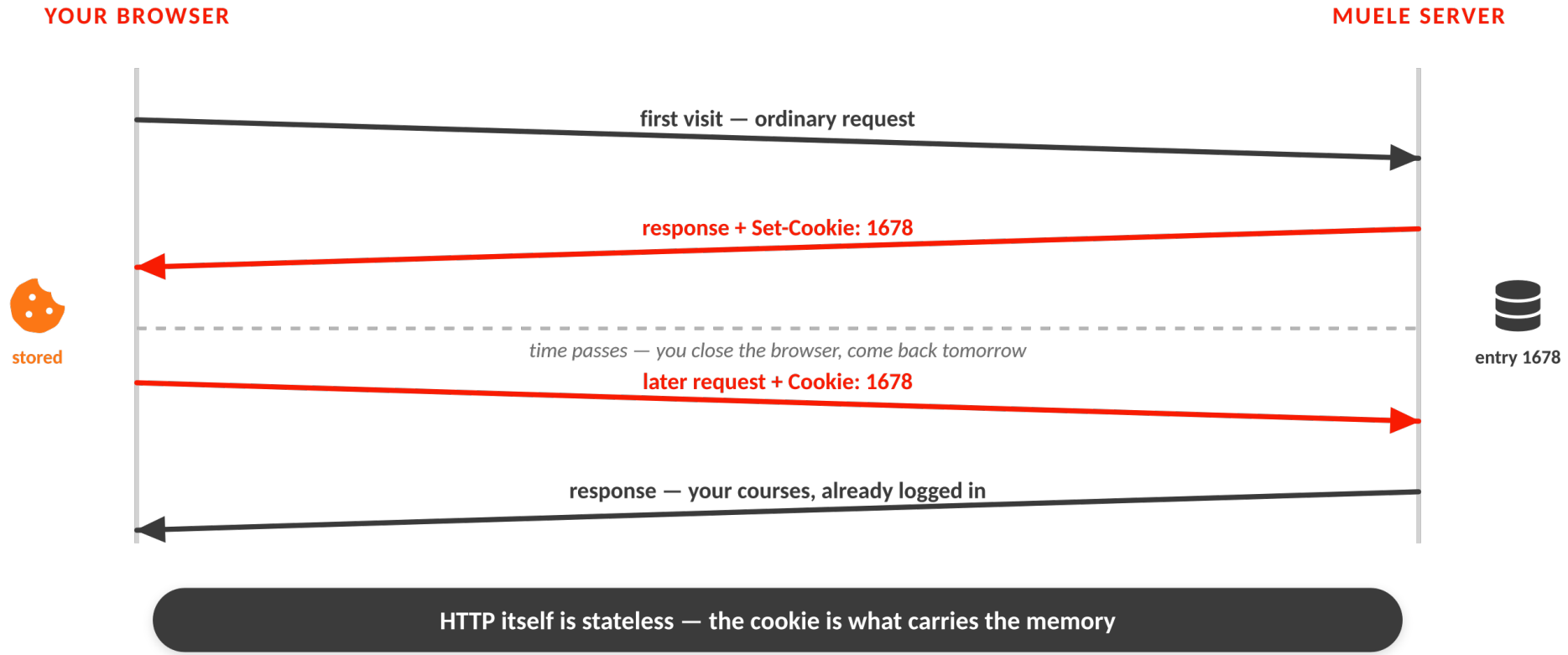
- The server remembers your connection
- Something is sent with every request
- The page is stored on your device
- HTTP is not really stateless



Memory and Speed

Cookies, caching, and why the web feels fast

Cookies: Giving a Stateless Protocol a Memory



Four pieces make it work: a Set-Cookie header in the response, a Cookie header in later requests, a file kept by your browser, and a record in the server's database.

Cookies Are Useful — and Uncomfortable



WHAT THEY MAKE POSSIBLE

- Staying logged in to MUELE as you move between pages
- A shopping basket that survives while you keep browsing
- Remembering your language or display preferences
- Not retyping your details on every single form

WHAT THEY ALSO ENABLE

- An advertising network can place a cookie on MANY different sites
- It then sees a slice of your browsing across all of them
- That builds a profile — without you ever giving your name
- Hence consent banners, and laws like the EU's GDPR

A cookie is just an identifier. Whether that is convenient or invasive depends entirely on who is collecting it, and how widely.

Web Caching: Keep a Copy Closer

A cache sits between the campus and the Internet, storing recently-requested objects so they need not be fetched twice.



Faster for the user

the reply comes from down the corridor, not across an ocean

Less traffic on the expensive link

Makerere's link to the ISP carries far less

Less load on the origin

the popular objects are served locally

How Much Does a Cache Help?

Imagine the campus link to the ISP is the bottleneck, and lecture slides are downloaded by many students on the same day.

ONE QUICK EXAMPLE

200 students each download the same **5 MB** slide deck.

Without a cache: 200 × 5 MB = **1,000 MB** crosses the campus link.

With a cache: the first request fetches 5 MB; the other 199 are served locally = **5 MB** crosses it.

A 99.5% reduction on the expensive link — and every student after the first gets it far faster.

Why this matters here: bandwidth to the wider Internet is expensive in Uganda. Caching is one of the cheapest ways to make a limited link feel much larger.

But What If the Copy Is Stale?



A cache must not serve an out-of-date page. The fix is to ask a cheap question before sending anything big.

```
Cache asks:   GET /notes.pdf   +   If-Modified-Since: Mon, 17 Aug 2026
```

304 Not Modified

Nothing has changed. The server sends this tiny reply with NO body at all — the cache serves its existing copy.

Cheap — a few bytes

200 OK + the file

The document was updated. The server sends the whole new version, and the cache replaces what it had.

Only when actually needed

QUICK CHECK · 3

A cache makes things faster. So why not cache absolutely everything, forever?

- Storage costs money
- The copy could go out of date
- Some pages are different per user
- The last two, mainly



04 DNS

How a name becomes an address

The Internet's Directory

You type a name. The network can only deliver to a number. Something has to translate — and it happens before any page loads.

`muele.mak.ac.ug`

what you type — easy to remember

DNS →

`102.134.147.141`

what the network needs — an IP address

Name → address

the core job, done before every connection

Aliases

one machine can answer to several names

Load spreading

one name can map to many servers

Mail routing

finds the mail server for a domain

Why Not Just One Big Directory?



Imagine a single machine holding every name on the Internet. It fails for four reasons — all of which you met in Chapter 1.



Single point of failure

if it goes down, the entire Internet stops resolving names



Traffic volume

every lookup on Earth hitting one machine — impossible



Distance

a server in one country would be far away from everyone else

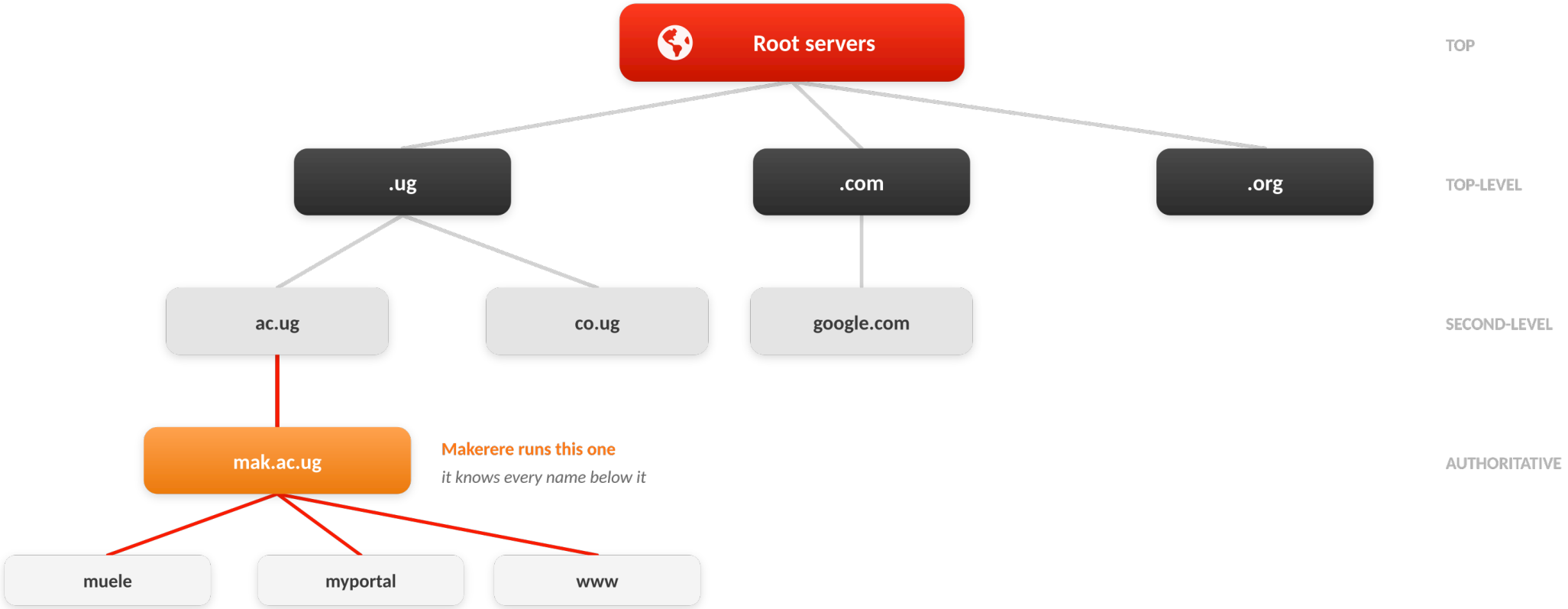


Maintenance

one database holding every change from every organisation

The answer is the same one the Internet always reaches for: do not centralise. Distribute the job, and let each organisation be responsible for its own names (problems).

A Hierarchy, Not a List



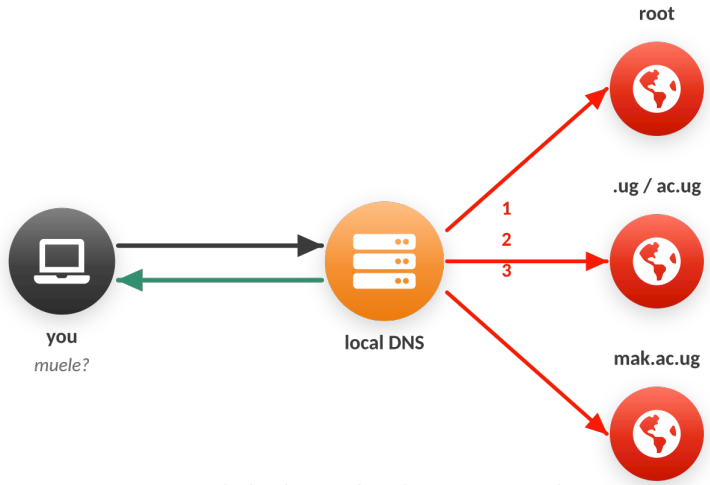
Read a name right to left: muele.mak.ac.ug — .ug is the country, ac.ug is academic institutions, mak.ac.ug is Makerere, and muele is one machine Makerere runs.

Two Ways to Run the Lookup

Your device asks one nearby server — the local DNS server. How it finds the answer is where the two styles differ.

ITERATED QUERY

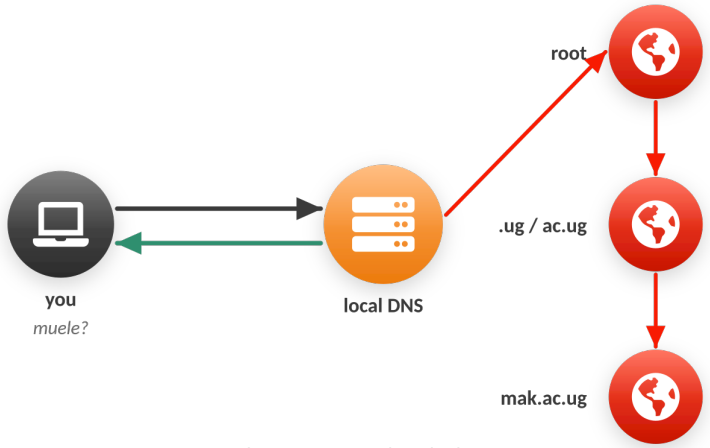
the usual way on the Internet



the local server does the running around

RECURSIVE QUERY

heavier load on the upper servers



each server passes the job along for you

In practice the Internet mostly uses iterated queries — it keeps the load off the root and TLD servers, which would otherwise be doing everyone's work.

Caching Again — Same Idea, Different Layer

That whole chain would be slow if it ran every time. It does not — answers are remembered along the way.

Your device

caches recent lookups — the fastest possible answer

Local DNS server

caches for everyone on campus — one lookup serves thousands

Time to live (TTL)

each record says how long it may be kept before re-checking

In practice: the first student to open MUELE each morning may trigger the full chain. Everyone after that gets the cached answer in a millisecond — which is why the very first page load of the day can feel slower.

QUICK CHECK · 4

Makerere changes the server behind muele.mak.ac.ug. Why might some students still reach the old one?

- Their browser cached the page
- A DNS record was cached with a TTL
- The old server refuses to stop
- DNS does not allow changes

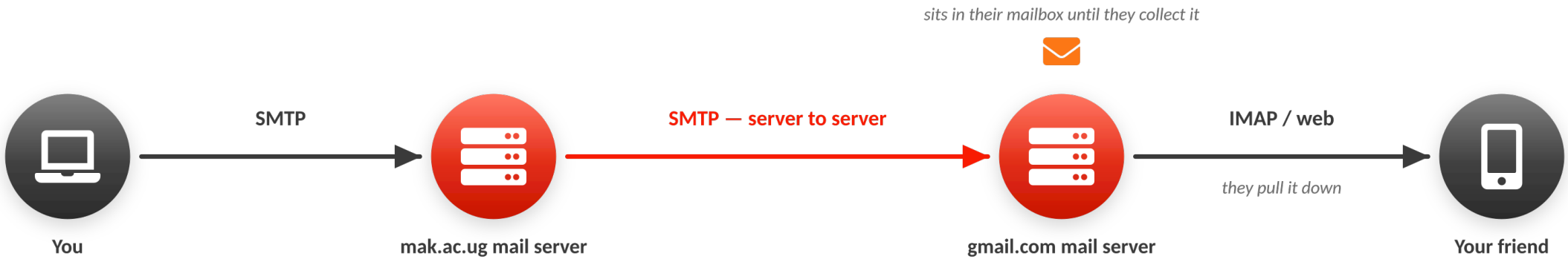


05 Email and Video

Two more applications, two different shapes

How an Email Actually Travels

Unlike the web, email is not one conversation — it is a relay, and the recipient need not be online when you send.



SMTP pushes mail toward the recipient; a separate protocol pulls it down

Note the asymmetry: SMTP PUSHES mail toward the recipient's server. But you PULL your own mail down when you open webmail — a different protocol for a different job.

SMTP Compared With HTTP

Both run over TCP and both use readable text commands — but they are built around opposite assumptions.

	SMTP (email)	HTTP (web)
Direction	Push — sender delivers to receiver	Pull — client fetches from server
Both ends online?	No — mail waits in a mailbox	Yes — both are present
Objects per message	Everything in one message	Each object fetched separately
Typical port	25 (and 587 for submission)	80 / 443 for HTTPS

Video: The Hardest Application

Video is most of the traffic on the Internet, and it stresses everything we studied in Chapter 1 at once.



Enormous volume

a single stream can need more bandwidth than hundreds of web pages



Must keep up

frames must arrive steadily — arriving late is as bad as not arriving



Networks vary

your available rate changes minute to minute, especially on mobile

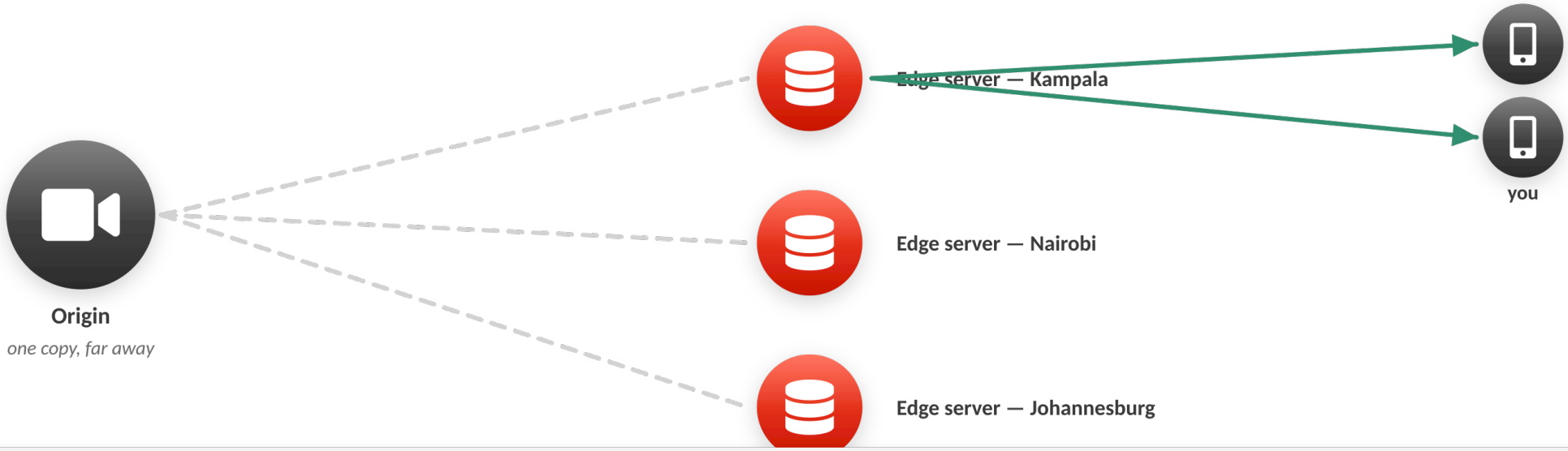
TWO FIXES YOU HAVE ALREADY SEEN IN ACTION

Buffering — download ahead of what you are watching, so brief slowdowns go unnoticed.

Adaptive quality — the video is stored at several quality levels, and your player switches between them as your connection changes. That is why the picture sometimes goes blurry, then sharpens again.

Content Delivery Networks

The last trick: do not send the video across the world at all. Put a copy near the viewer in advance.



The video is copied to servers near you in advance.

You stream from the closest copy — shorter path, less delay, fewer stalls.

QUICK CHECK · 5

Caching, CDNs, DNS caching — what single idea keeps reappearing?

- Keep a copy closer to the user
- Compress everything
- Use faster cables
- Send fewer requests

WRAPPING UP

Key Takeaways



Applications

run only on hosts, use sockets to reach the network, and are addressed by IP plus port

Transport choice

TCP when nothing may be lost, UDP when being late is worse than being lost

HTTP

a stateless request–response protocol; persistent connections avoid paying RTT repeatedly

Cookies

give a stateless protocol a memory — useful, and the basis of tracking

DNS

a distributed hierarchy that turns names into addresses before anything else happens

Caching

the recurring idea: a copy closer to the user, with a rule for staying fresh

WRAPPING UP

Where We Go From Here

We have been standing on top of a service we simply assumed worked. Next we open it up.

1	**Transport Layer**	How TCP actually delivers reliably over an unreliable network — and what it costs
2	**Network Layer**	How routers choose a path, and how IP addresses are structured
3	**Link & Physical**	What happens on a single hop — Ethernet, Wi-Fi, and the hardware beneath

Also coming soon: a lab session where you write the socket code yourself — a small client and server of your own.

Questions?

Next: the Transport Layer — how TCP keeps its promise.